



조현준.

Systems & AI Full-Stack Engineer | BUAA · SW Engineering · 2026.07 졸업 예정

BUAA Software Engineering

TSC 중국어 Lv.6

OPIc ENG IH

한·중·영 커뮤니케이션

중국 소프트웨어공학 전국 1위 BUAA(북항대)에서 8년. OS 커널을 밑바닥부터 직접 구현하고, ECG AI 모델 AUC 92.52%를 달성한 시스템·AI 엔지니어입니다. 저수준 시스템부터 풀스택 서비스까지 End-to-End로 설계하고 구현합니다.

2026.07

졸업 예정

TSC 6

중국어 레벨

OPIc IH

영어 회화

한·중·영

업무 소통

I 학력 & 어학

ED 학력

北京汇文中学

북경 회문고등학교

2017.09 - 2020.07 | 150년 전통 북경 동성구 명문 공립고교

北京航空航天大学 (BUAA)

软件学院 (School of Software) · 학사

2020.09 - 2026.07 (졸업 예정) | 소프트웨어공학 전국 1위 (A+)

LG 어학 & 언어 능숙도

TSC 중국어 회화 Level 6

CN

원어민 수준 비즈니스 커뮤니케이션 | ID: 100318 · 2025.10.25

OPIc 영어

US

IH · Intermediate High

업무 환경 유창한 영어 소통 | ID: 2C5955731527 · 2025.12.22



I 나에 대해

RD

학습 자세

막히면 포기 대신 **가설을 세워 정면 돌파**합니다. 새로운 기술은 공식 문서부터 직접 읽고 구현하며 **내재화**하고, OS 커널을 바닥부터 디버깅했던 집요함으로 **해결될 때까지 검증**을 반복합니다.

가설 검증형

문서화 습관

TM

협업 방식

기술 스택 충돌을 **데이터로 중재**합니다. 성능·생태계 데이터를 직접 수집해 팀을 설득하며, 합의 내용은 **RFC 문서로 기록**합니다. **한·중·영**으로 팀 전체가 같은 맥락을 공유합니다.

한·중·영

RFC 문서화

데이터 기반

CL

취미 & 라이프스타일

암벽 클라이밍을 즐깁니다. 최적의 루트를 분석하고 완등에 도전하는 과정은 **엔지니어링 문제 해결**과 닮아있습니다. 일상에서도 **집요하게 길을 찾는 태도**를 유지합니다.

클라이밍

루트파인딩

I 기술 스택

● 시스템 / 로우 레벨

C / C++

Java

MIPS Assembly

OS Kernel

Compiler

● 백엔드 / 데이터베이스

Spring Boot 3

PostgreSQL

Elasticsearch

Redis

● 프론트엔드 (웹 / 모바일)

React Native

Vue 3

TypeScript

● AI & ML

Python

CNN-CBAM-GRU

FastAPI

● 인프라 / 협업

Docker

MinIO

Git

RFC Docs

GitHub Actions

AWS EC2

LANG **6** 언어

FW **4** 프레임워크

OPS **6** 인프라 도구

AI **2** AI 모델 구현

01

BUAA OS — MIPS 마이크로커널 운영체제

2024.03 - 2024.07

BUAA OS Course

OS Kernel Developer · 1인 개발

GXemul MIPS 시뮬레이터 위에서 커널 부팅 · 가상메모리 · CoW fork · 마이크로커널 FS를 7단계로 직접 구현

✓ Lab 0-6 전 단계 완주

GXemul 환경에서 Lab0~Lab6의 7단계 커리큘럼을 따라 커널 부팅·메모리 관리·프로세스·시스템콜·IPC·파일시스템까지 OS의 핵심 구성요소를 단계적으로 직접 구현했습니다. 특히 **TLB Fast Refill**, **Copy-on-Write** 기반 **fork()**, 마이크로커널 파일시스템 구조를 구현하며 운영체제 핵심 메커니즘을 깊이 실습했습니다.

| 주요 개발 내용 (Lab 0 - Lab 6)

- 커널 부팅 및 메모리 초기화 (**Lab0-1**): GXemul 부트스트랩 코드 작성, 물리 메모리 레이아웃 설정 및 커널 엔트리 포인트 초기화.
- 가상 메모리 (**Lab2**): 2단계 페이지 테이블 기반 가상 주소 공간 설계, **TLB Miss Fast Refill** 핸들러를 어셈블리로 직접 작성해 TLB 관리 오버헤드 최소화.
- 프로세스 스케줄링 (**Lab3**): PCB 설계, 라운드 로빈 스케줄러 구현, 컨텍스트 스위칭 시 레지스터 상태 저장·복원 로직 직접 작성.
- **Copy-on-Write fork()** (**Lab4**): 페이지 공유 후 쓰기 시 예외 기반 실제 복사가 일어나는 COW 메커니즘 적용, 지연 평가(Lazy Evaluation)로 메모리 복사 비용 최적화.
- **Pipe IPC + 마이크로커널 FS (Lab5-6)**: 공유 페이지 기반 파이프 구현 및 독립된 **FS** 서버 프로세스와 **IPC**로 통신하는 마이크로커널 구조 구현.

Pipe IPC — pageref

Kernel / IPC

판정 레이스 컨디션

해결

상황

`dup()` · `close()` 실행 시, FD와 Pipe 페이지의 **매핑·해제 순서**에 따라 **pageref** 값이 일시적으로 의도와 다르게 관측되는 레이스 컨디션 발생.

대안 탐색

인터럽트 비활성화로 임계 구간 보호 검토 → 성능 저하·설계 원칙 위배로 제외. **매핑 순서 재배치(Reordering)**로 불변식 유지 방안 선택.

해결

`pageref(fd) ≤ pageref(pipe)` 조건이 어떤 순간에도 깨지지 않도록 **unmap · map** 순서를 조정했습니다.

pipe.c - Atomic-like Mapping

```
// [Fix] FD를 먼저 해제하여
// pageref(fd) > pageref(pipe) 찰나의 순간 방지
void pipe_close(int fdnum) {
    syscall_mem_unmap(0, fd_to_va(fdnum));
    syscall_mem_unmap(0, pipe_to_va(fdnum));
}
```

알려진 점

선점형 스케줄링에서는 '완성된 결과'뿐만 아니라 '**과정 중의 중간 상태**'도 **안전해야 함**을 깨달았습니다. 불변식 보장을 코드 레벨에서 강제하는 습관을 만들었습니다.

C

MIPS Assembly

GXemul Simulator

Microkernel Architecture

IPC (Pipe)

Virtual Memory · TLB

CoW fork()

File System Server

02

SysY 언어 컴파일러 프론트엔드 & VM 구현

2024.10 - 2025.01

BUAA Compiler Course

Compiler Engineer · 1인 개발

Lexer → 재귀 하향 Parser → Symbol Table → P-Code VM, 컴파일러 전체 파이프라인을 C++ 으로 구현

✓ a-m 전 에러 범주 처리 달성

SysY 언어의 어휘 분석기(Lexer)부터 재귀 하향 파서(Parser), 심볼 테이블, P-Code 생성기, 가상 머신(VM) 인터프리터까지 컴파일러 전체 파이프라인을 구현했습니다. 초기 단일 파일 구조에서 `Lexer`, `Parser`, `SymbolTable`, `ErrorHandler`, `Interpreter` 모듈로 분리하며 객체지향적 설계와 유지보수성을 개선했습니다.

| 주요 개발 내용

- 어휘 분석기(Lexer): `unordered_map`으로 키워드 판별 최적화, `peek()` 메커니즘으로 `<=` · `==` · `&&` 등 다중 문자 연산자와 이스케이프 시퀀스를 일관되게 토큰화.
- 재귀 하향 파서(Parser): `CompUnit` 선행 탐색, `LVal = Exp` vs `[Exp]` 모호성 판별, 연산자 우선순위 기반 계층적 식(Expression) 파싱 구현.
- 스코프 기반 심볼 테이블: 스코프 ID + 스택 구조로 변수 새도잉, 함수 시그니처 불일치, 상수 재대입, 미선언 식별자 등 의미 오류를 컴파일 타임에 검출.
- Fault-Tolerant 에러 복구: 에러를 파싱 중단 사유가 아닌 '문맥 복귀 신호'로 재설계. `mLastNonT` 기준 복구로 연쇄 오류 출력 방지, 파싱 지속.
- P-Code VM: 산술·논리·분기·반복·함수 호출/반환·배열 인덱싱·I/O에 대한 자체 P-Code 명령어 셋 설계 및 스택 머신 인터프리터 구현.

Fault-Tolerant 파서 — 연쇄 파싱 에러 및 중복 오진 문제 해결

Compiler / Parser

상 황

`int a int b;` 처럼 세미콜론 누락 케이스를 처리하려고 에러 함수를 추가하면, 기존에 통과하던 정상 파싱까지 깨지는 문제 반복. `parseErrorI/J/K`가 `mCurToken`을 직접 소비하는 함수여서, 호출 순서가 조금만 바뀌어도 후속 구문이 엉뚱한 토큰에서 시작했습니다.

해 결

에러 함수를 토큰 소비형 / look-ahead형 /

Parser.cpp - Error Recovery

```
// parseErrorI/J/K 는 토큰 소비 함수
bool Parser::parseErrorI() {
    if (mCurToken.word != ";") {
        m_pErrorHandler->PushErrorList(
            'i', mLastNonT); // 마지막
        위치 기준
        return true;
    }
    AdvanceToNextToken(); // ← 소
    비
    return false;
}
```

알 게 된 점

문제 해결 시 재현 가능한 최소 입력을 만들고, 상태를 바꾸는 함수와 읽기만 하는 함수를 구분해야 한다는 점을 배웠습니다. 에러 처리는 예외 출력보다 파서의 현재 위치·문맥 보존 설계가 더 중요했습니다.

C++17

CMake

Lexer / Tokenizer

Recursive Descent Parser

Symbol Table · Scope Chain

P-Code VM · Stack Machine

Fault-Tolerant Error Recovery

03

학술 성과 공유 플랫폼 (Academic Results Sharing Platform)

2025.03 - 2025.08

BUAA SSAD Course

Backend / Full-Stack · 팀 프로젝트

연구자 학술 성과 검색·분석 플랫폼 — ES · Redis · MinIO · Vector 5개 인프라 통합, 백엔드 전담

✓ API 응답 규약 통일로 프론트 파싱 오류 제거

Vue 3 SPA + Spring Boot + PostgreSQL + Elasticsearch + Redis + MinIO + Vector Service를 연동하는 학술 성과 검색·분석 플랫폼입니다. 백엔드 전담으로 인프라 통합 레이어 설계, 공통 API 응답 규약 정의, 프론트엔드-백엔드 인터페이스 연동을 담당했습니다. docs/ 및 RFC/를 기반으로 전 과정의 문서를 명문화했습니다.

주요 개발 내용

- 다중 인프라 통합 레이어: Spring Boot 3 (Java 21) 환경에서 PostgreSQL 18 · Elasticsearch · Redis · MinIO · Vector Service 5개 인프라를 하나의 백엔드로 통합 설계 및 구현.
- 공통 API 응답 규약: `ResponseBodyAdvice`를 통한 응답 자동 래핑 (`code/message/data`) 및 `BusinessException` 상속 체계 기반의 전역 예외 처리 로직 구현 → 프론트 파싱 오류 제거.
- **Controller** 이중 레이어 구조: 계층 A(account · general · search · output) + 계층 B(scholar · analyze · announcement · storage · update)로 관심사 분리.
- 스케줄러 / **Background Tasks**: metrics update · hot point sync · summary refresh를 비동기 스케줄러로 구현, 서비스 응답성 유지.
- **RFC** 문서화 체계: 팀 합의 내용을 RFC 문서로 기록, 한·중·영 3개국어 주석으로 팀 전체 컨텍스트 동기화.

백엔드 실행 환경 파편화

백엔드 / 협업

및 API 응답 규약 불일치 해결

상황

팀원 간 로컬 환경 설정 (.env · AI 모델 경로) 차이로 인한 **인프라 기동 실패** 발생.
개발자별로 상이한 API 응답 구조로
프론트엔드에서 데이터 파싱 오류가
빈번하게 나타났습니다.

해결

README 기반 기동 프로세스를 관리하고

● ● ● ResponseResultHandler.java

```
@RestControllerAdvice
public class ResponseResultHandler
    implements ResponseBodyAdvice<Object> {

    @Override
    public Object beforeBodyWrite(Object body,...) {
        if (body instanceof ResultWrapper)
            return body;
        return ResultWrapper.success(body);
    }
}
```

알려진 점

협업 시 휴먼 에러를 줄이려면 구두 약속보다
**시스템(아키텍처)적으로 규약을 강제하는
설계가** 팀 생산성에 더 크게 기여함을
체감했습니다.

Spring Boot 3

PostgreSQL 18

Elasticsearch

Redis

MinIO · Vector Service

Vue 3 · TypeScript

Docker Compose

OpenAlex / ORCID API

04

CareLink — 노인 건강 모니터링 & AI ECG 진단 플랫폼

논문

2025.10 - 2026.03

3-Tier 시스템 통합 및 AI 모델 구축 · 1인 개발

모바일(React Native) · 백엔드(Spring Boot) · AI 추론(FastAPI) 3-Tier를 단독 설계 — 환자-보호자 연결 + ECG 자동 진단

✓ ECG AUC 92.52% · TPS 66 · AWS 배포

환자와 보호자를 연결하는 실시간 헬스케어 플랫폼입니다. **3-Tier** 분리 구조(비즈니스 로직 + AI 고부하 연산 분리)를 단독 설계하고 구현했습니다. PTB-XL 기반 ECG 분류 모델(**CNN-CBAM-GRU**) AUC 92.52% 달성, FastAPI 추론 서버와 통합해 실시간 진단 기능을 구현했습니다.

I 주요 개발 내용 & 성능 지표

→ **3-Tier** 아키텍처: React Native / Spring Boot 3 / FastAPI + PyTorch. 비즈니스 로직과 AI 고부하 연산을 컨테이너 레벨에서 분리해 운영 안정성 확보. AWS EC2 + Docker Compose 배포.

→ **ECG 진단 모델 (CNN-CBAM-GRU)**: PTB-XL 21,837 records · 12-lead · 500Hz. 5개 슈퍼클래스 멀티라벨 분류. **Macro AUC 92.52%** · Macro F1 74.52% · 0.65M 파라미터. 비대칭 손실 + pos_weight 클래스 불균형 보정.

→ **TPS 성능**: 20명 동시 접속 부하 테스트에서 **TPS 56.23~66.41**. 추론 속도 평균 2,382ms · P95 2,698ms.

→ **Cache-First 전략**: AsyncStorage 기반 적용 후 Home 첫 표시 평균 **0.29ms**, 네트워크 동기화 완료 평균 604ms.

→ **JWT 인증 + 보호자 연결**: 401 만료 자동 로그아웃 흐름 구현, 보호자-환자 실시간 연결 API 설계.

Ubuntu EC2

Ubuntu / Docker Logs

로그 추적으로

ECG 분석

API 500 원인

해결

① 증상

`POST /api/ecg/analyze` 가 간헐적으로 500 반환. 백엔드 로그에

`/predict_window` 요청이 **Connection refused**로 실패 기록.

② 추적 · 원인

`docker logs carelink-ai` 로 AI

● ● ● Ubuntu EC2 — Log Trace

```
# 1. 백엔드→AI 호출 실패 확인
docker logs carelink-backend --tail=100
```

```
I/O error on POST http://ai:8000/predict_window
```

```
# 2. AI 컨테이너 종료 원인
```

```
docker ps -a | grep carelink-ai
docker logs carelink-ai --tail=50
```

```
# 3. OOM 여부 커널 로그 확인
```

```
sudo journalctl -k | grep "out of memory"
```

알려진 점

문제 해결 우선순위: **증상 재현** → **앱 로그** → **컨테이너 상태** → **OS 커널 로그** 순서로 좁히는 습관을 만들었습니다. API 500도 코드만 보지 않고 인프라 자원까지 확인해야 했습니다.

React Native · Expo

Spring Boot 3

FastAPI · PyTorch

CNN-CBAM-GRU

PostgreSQL

AWS EC2

Docker

GitHub Actions